

Assignment 3 | Bridge Pattern Report

Student: Muratbek Akyl

Group: SE-2523

Topic: Option A (Drawing)

Course: ShP-2216 Software Design Patterns

Institution: Astana IT University

Base Commit:

c178fa274853e3a96ddf2202e09df605497ec72b

Repository URL: https://github.com/muratbek-akyl/SDP_Assignment3

Submitted Commit: ebcd82e2d20040260bfaac4583009eaa0235f229

1. Architecture and Problem Domain

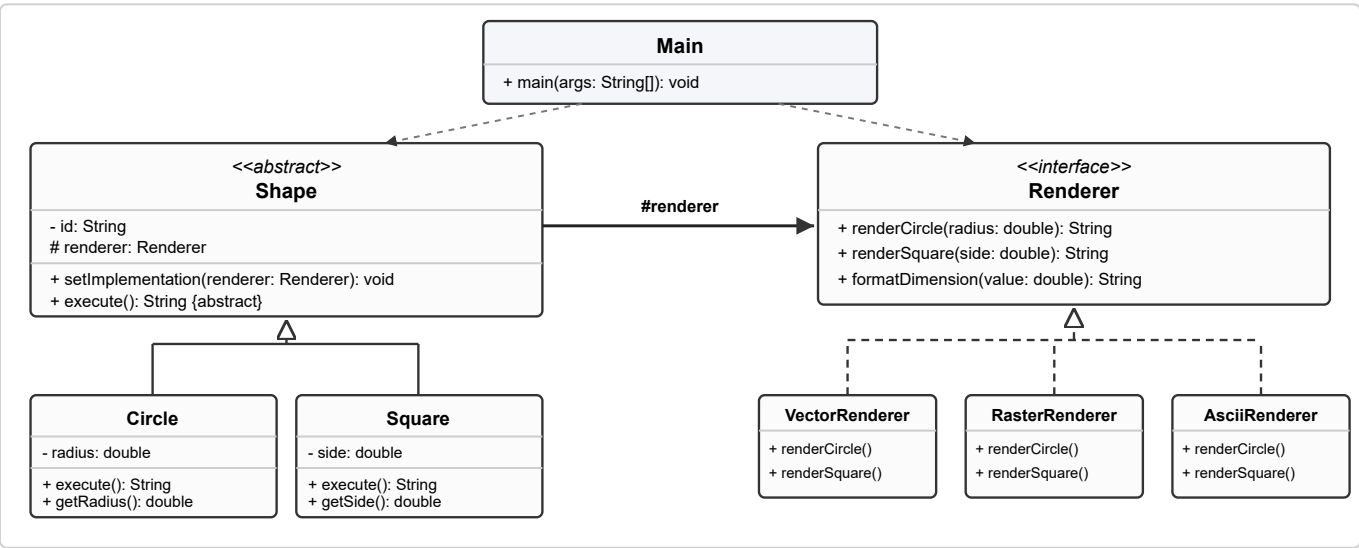
The objective of this assignment is to decouple an abstraction from its implementation using the Gang of Four (GoF) Bridge pattern, allowing both hierarchies to vary independently without class explosion. In Option A (Drawing), the application models geometric entities that need to be rendered across different rendering formats.

Two independent dimensions of variation are identified:

- Domain Abstraction Dimension (Shape Hierarchy):** Represents geometric concepts, namely Circle (defined by radius 2) and Square (defined by side 3). This side maintains domain state (unique ID, dimensions) and exposes high-level operations via execute() and runtime reconfiguration via setImplementation(Renderer renderer).
- Technical Implementation Dimension (Renderer Hierarchy):** Represents low-level rendering strategies, namely VectorRenderer, RasterRenderer, and AsciiRenderer. This side defines primitive drawing contracts (renderCircle, renderSquare) and produces target string representations.

Why Bridge Fits: In a naive inheritance hierarchy, combining M shapes with N renderers produces $M \times N$ concrete classes (e.g., VectorCircle, RasterCircle, AsciiCircle, VectorSquare, RasterSquare, AsciiSquare). Adding a new shape or renderer causes exponential class proliferation. Bridge replaces inheritance with object composition: Shape holds a reference to the Renderer interface. This reduces total classes to $M + N$ and allows runtime swapping of renderers on the same shape instance without recreating the object or losing domain state.

2. UML Class Diagram



3. Clean Code Practice Excerpts

The implementation strictly adheres to clean code principles without comments. Below are five annotated excerpts illustrating these practices:

1. Separation of Responsibilities

The Shape abstraction hierarchy manages domain state and delegates rendering, while concrete renderers focus solely on formatting output.

```
public class Circle extends Shape {
    @Override
    public String execute() {
        return renderer.renderCircle(radius);
    }
}
```

Benefit: High cohesion and single responsibility. Circle has no knowledge of string concatenation or drawing formats; changing rendering logic does not touch geometric calculations or shape definitions.

2. Meaningful Names

All types, methods, and parameters clearly convey their intent and design pattern role without abbreviations or obscure acronyms.

```
public class VectorRenderer implements Renderer {
    @Override
    public String renderCircle(double radius) { ... }
}
```

Benefit: Readability is self-contained. Any developer instantly grasps what VectorRenderer produces and that it satisfies the Renderer contract, removing the necessity of explanatory comments.

3. Small Focused Methods

Every method performs a single distinct task with minimum cognitive load and no hidden side effects.

```
@Override
public String renderSquare(double side) {
    return "ASCII square side=" + formatDimension(side);
}
```

Benefit: The method is concise and predictable. It accepts domain input, relies on utility formatting, and produces a deterministic string, making it trivial to test and verify.

4. Avoidance of Duplication (DRY Principle)

Common numeric formatting logic is defined once as an interface default method, and test evaluation logic in Main is unified into a single helper method.

```
public interface Renderer {
    default String formatDimension(double value) {
        if (value == (long) value) {
            return String.valueOf((long) value);
        }
        return String.valueOf(value);
    }
}
```

Benefit: Eliminates code duplication across all three renderers (VectorRenderer, RasterRenderer, AsciiRenderer). Modifying number representation requires changing only one centralized method.

5. Encapsulation of State

Internal fields are declared private final with explicit public read accessors. The bridge reference can only be modified via an explicit setter with validation.

```
public abstract class Shape {
    private final String id;
    protected Renderer renderer;

    public void setImplementation(Renderer renderer) {
        this.renderer = Objects.requireNonNull(renderer, "renderer must not be null");
    }
}
```

Benefit: Domain data (such as shape ID and dimensions) remains immutable after instantiation. Implementor replacement is controlled, guarding against invalid null references.

4. Independent Extension and Trade-off Analysis

Independent Extension (I3 Addition)

To demonstrate the Open/Closed Principle and the decoupling provided by Bridge:

- The third concrete implementor, `AsciiRenderer`, was introduced to format shapes using ASCII text.
- Only `AsciiRenderer.java` was added, and `Main.java` was updated to run checks T6 and T7.
- **What Stayed Unchanged:** The existing abstraction classes (`Shape`, `Circle`, `Square`), the `Renderer` interface contract, and the existing implementors (`VectorRenderer`, `RasterRenderer`) remained completely untouched.

This is verified by `extension.diff`, confirming that the extension required zero modifications to the existing abstraction or implementor hierarchies.

Trade-off of the Bridge Pattern

The primary trade-off of Bridge is **increased architectural complexity and indirection**. Instead of directly calling a method on an object, the abstraction introduces an extra layer of delegation through an interface. For small or static projects where only one rendering mechanism is ever used, introducing Bridge creates unnecessary boilerplate and interface management. However, for systems where platforms, drivers, or formats vary dynamically, this trade-off is well worth the decoupling benefits.

Bridge vs. Adapter Pattern Comparison

While both patterns rely on object composition to forward requests, their intent, lifecycle, and design phases differ fundamentally:

- **Design Phase and Intent:**
 - **Bridge:** Applied *upfront during initial system design*. Its goal is to proactively separate an abstraction from its implementations so both hierarchies can evolve independently over time.
 - **Adapter:** Applied *retrospectively after components exist*. Its goal is to bridge the gap between two incompatible, pre-existing interfaces without modifying their underlying source code.
- **Application to this Project:** In our application, `Shape` and `Renderer` were intentionally designed together as collaborative abstractions. If we later had to integrate a closed-source third-party graphics library with a method like `drawRawCircle(int x, int y, int r)` that does not match our `Renderer` interface, we would write an `Adapter` wrapping that third-party class to conform to `Renderer`.

5. References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
3. Bekenov, Y. (2026). *Software Design Patterns: Lecture 4 - Bridge Pattern*. School of Software Engineering, Astana IT University.